Dal latch al registro a scorrimento

Lezione 3 — 2MAK, SST-MEC-25c — Logica sequenziale

24/09/2026

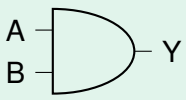
1 Ripasso: le porte logiche di base

Definizione

Le **porte logiche** viste finora sono **AND**, **OR**, **NOT**, e le loro combinazioni **NAND** (AND + NOT in uscita) e **NOR** (OR + NOT in uscita). Tutte e cinque restituiscono un'uscita che dipende **esclusivamente** dal valore attuale degli ingressi: questo tipo di circuito si chiama **combinatorio**.

Esempio

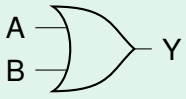
Porta **AND**: l'uscita vale 1 solo se **entrambi** gli ingressi valgono 1.



A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Esempio

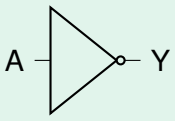
Porta **OR**: l'uscita vale 1 se **almeno uno** dei due ingressi vale 1.



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Esempio

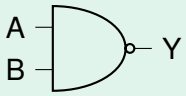
Porta **NOT**: un solo ingresso, l'uscita è sempre l'opposto dell'ingresso.



A	Y
0	1
1	0

Esempio

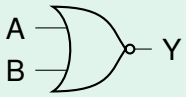
Porta **NAND**: una AND seguita da una NOT in uscita (il pallino sull'uscita indica la negazione). L'uscita è l'**opposto** di quella di una AND.



A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

Esempio

Porta **NOR**: una OR seguita da una NOT in uscita. L'uscita è l'**opposto** di quella di una OR.



A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

Nota

NAND e NOR sono le porte più usate nella costruzione di circuiti ad alta densità (es. microprocessori): richiedono meno transistor delle porte "pure" AND/OR (tipicamente 4 transistor per porta), quindi sono più economiche, più piccole e permettono di integrarne di più nello stesso spazio. Un moderno **processore** contiene circa 10 miliardi di **transistor**; un processore degli anni '80 (es. un 286) ne aveva nell'ordine delle 10.000 porte logiche.

Esempio

Quando si scrive in un programma un'operazione come `5 + 3`, non esiste un software che "capisce" l'addizione: viene attivato un **circuito combinatorio** specifico, fatto di porte logiche, che esegue quel calcolo. Un microcontrollore semplice, senza un circuito dedicato per la moltiplicazione, deve simulare una moltiplicazione con una sequenza di somme ripetute: per questo un'operazione complessa richiede più tempo di una semplice.

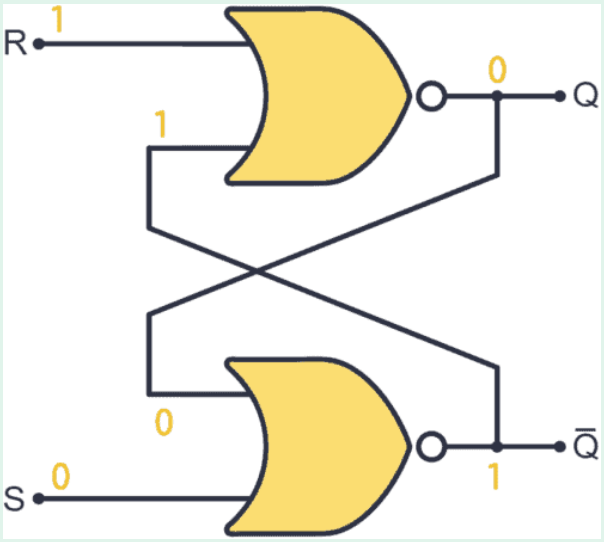
2 Il latch SR: la prima cella di memoria

Definizione

Un circuito **combinatorio** ha l'uscita che dipende solo dagli ingressi attuali. Un circuito **sequenziale** ha invece un'uscita che dipende anche dallo **stato precedente**: per ottenerlo, si porta l'**uscita** di una porta a richiudersi sul proprio ingresso. Questo collegamento a ciclo si chiama **retroazione**.

Esempio

Il **latch SR** (Set-Reset) si costruisce con due porte **NOR** incrociate: l'uscita della prima porta è collegata a un ingresso della seconda, e viceversa.



Con $S = 0$ e $R = 0$, il circuito **mantiene** l'ultimo stato: se prima era $Q = 1$, resta $Q = 1$; se era $Q = 0$, resta $Q = 0$. Questa è la proprietà di **memoria**.

Definizione

Tabella di verità del **latch SR** (con NOR incrociate):

S	R	Q	$\overline{Q}$
0	0	mantiene	mantiene
1	0	1	0
0	1	0	1
1	1	*stato non ammesso*	

Attenzione

La combinazione $S = 1, R = 1$ **non è ammessa**: forzerebbe **entrambe le uscite a 0**, violando la relazione $Q = \overline{\overline{Q}}$ che il circuito dovrebbe sempre rispettare (le due uscite devono essere **sempre l'una il negato dell'altra**). Va sempre evitata.

Nota

I nomi degli ingressi non sono casuali: **Set** significa "imposta" e **Reset** significa "cancella". Portando $S = 1$ si impone (si salva) un 1 in uscita; portando $R = 1$ si **impone** (si cancella) uno 0. Con $S = R = 0$ non si dà nessun comando nuovo, e il circuito si limita a mantenere l'ultimo valore che gli è stato imposto.

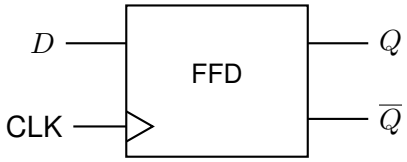
Nota

Il latch SR è stato uno dei primissimi dispositivi di **memoria a stato solido**. Prima, i computer usavano sistemi meccanici come le **memorie a nuclei di ferrite** (*core memory*): una griglia di fili orizzontali e verticali con un anellino di materiale magnetizzabile a ogni incrocio; la presenza o assenza di magnetizzazione in quell'anellino rappresentava un **bit**. Un armadio di questo tipo, grande quanto un frigorifero, conteneva pochi kilobyte di memoria.

3 Il flip-flop di tipo D

Definizione

Esistono varianti più sofisticate del latch, che impediscono di raggiungere lo **stato non ammesso**. Quella che useremo per il resto dell'anno è il **flip-flop di tipo D** (sigla **FFD**): un **elemento di memoria** con due ingressi (**D**, dato; **CLK**, clock) e due uscite (Q, $\overline{Q}$).



Esempio

Il flip-flop D funziona come una macchina fotografica: l'ingresso D può cambiare liberamente nel tempo, ma questo non ha alcun effetto sull'uscita. Solo nell'istante in cui il segnale di clock passa da basso ad alto (un **fronte di salita**), il flip-flop "**fotografa**" il valore che D ha in quel preciso momento e lo riporta in uscita su Q, dove resta fino al fronte di salita successivo.

Attenzione

Il flip-flop reagisce solo al fronte di salita del clock, mai al fronte di discesa (quando il clock passa da alto a basso) e mai al semplice valore assoluto del clock. Un **fronte di discesa** non provoca alcun aggiornamento dell'uscita.

4 Il contatore asincrono

Definizione

Collegando più flip-flop D **in cascata** – l'uscita $\overline{Q}$ di ciascuno collegata al proprio ingresso D, e il **clock** di ciascuno stadio pilotato dall'uscita $\overline{Q}$ dello stadio **precedente** (solo il primo flip-flop riceve il clock "vero") – si ottiene un **contatore asincrono**: a ogni **fronte attivo** del segnale che lo pilota, ciascun flip-flop avanza al valore successivo, e il cambiamento si propaga in cascata da uno stadio al successivo. Si chiama **asincrono** perché i flip-flop non cambiano **tutti** nello **stesso istante**: ognuno aspetta che sia cambiato quello prima di lui, introducendo un piccolo **ritardo di propagazione** stadio dopo stadio.

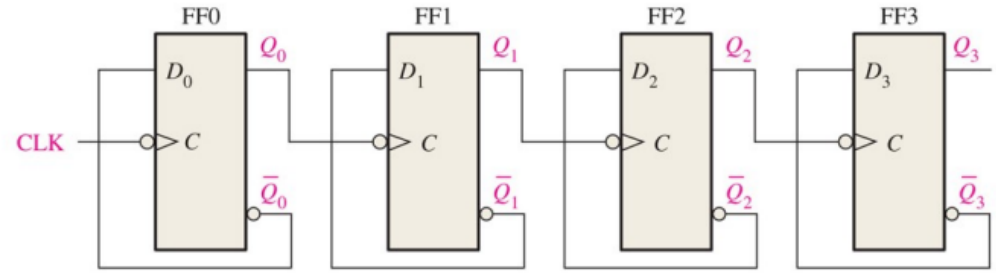
Esempio

Contatore asincrono a **4 bit** con 4 flip-flop D (**FF0, FF1, FF2, FF3**), ciascuno **negativo-triggerato** (scatta sul fronte di **discesa**). Il **CLK** esterno pilota **solo FF0**; da lì' in poi, l'uscita $\overline{Q}$ di ciascuno stadio diventa il **clock dello stadio successivo**. In ogni stadio $\overline{Q}$ è anche ricollegato al proprio D (come nel contatore visto in teoria), cosicché ogni flip-flop si inverte a ogni proprio fronte di clock.

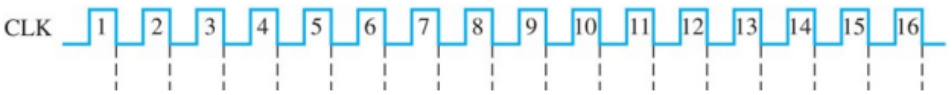
A 4-bit asynchronous binary counter is shown below. Each D flip-flop is negative edge-triggered.

(a) Develop a timing diagram showing the Q output of each flip-flop.

(b) Redesign the circuit for a decade counter.



(a)



Le uscite Q_0, Q_1, Q_2, Q_3 formano il numero binario contato (con Q_0 bit meno significativo): a ogni fronte di discesa del CLK esterno, FF0 scatta e Q_0 si inverte, quindi anche $\overline{Q}_0$ si inverte; quando $\overline{Q}_0$ ha il suo fronte di discesa (cioè quando Q_0 passa da 0 a 1), FF1 scatta e Q_1 si inverte; e così via a cascata sugli stadi successivi.

Esempio

Partendo da 0000, ogni colpo di clock fa avanzare il conteggio: $0000 \to 0001 \to 0010 \to 0011 \to 0100 \to \ldots$ La prima uscita (Q_0) cambia a ogni colpo di clock, la seconda (Q_1) ogni 2 colpi, la terza (Q_2) ogni 4 colpi, la quarta (Q_3) ogni 8 colpi: ciascuno stadio **dimezza la velocità** di cambiamento rispetto al precedente – esattamente come in un contatore sincrono, ma qui il cambiamento si **propaga stadio dopo stadio** invece di avvenire tutto insieme.

Nota

Per leggere il numero contato bisogna saper convertire tra **base 2** e **base 10**. Vale la **notazione posizionale**: il bit più a destra vale 2^0, il successivo 2^1, poi 2^2, 2^3, e così via (ricordando che qualunque numero **elevato a 0** vale 1). Ad esempio il numero binario 10010 si converte così:

$$1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 16 + 0 + 0 + 2 + 0 = 18$$

Per il verso opposto (da **decimale a binario**) si **divide ripetutamente per 2** annotando i resti, e poi si leggono i resti dall'ultimo al primo: dividendo 18 per 2 più volte di seguito (18, 9, 4, 2, 1) si ottengono i resti $0, 1, 0, 0, 1$, che letti al contrario ricostruiscono 10010.

Nota

Un contatore di questo tipo è alla base del concetto di **program counter** nei **microprocessori**: il registro che tiene traccia di quale riga di codice deve essere eseguita. La macchina esegue le istruzioni una alla volta, e a ogni passo il program counter avanza al successivo. La velocità

con cui il program counter avanza è legata alla **frequenza di clock** del processore: più veloce il clock, più rapidamente si eseguono le istruzioni. (I program counter reali dei microprocessori sono quasi sempre **sincroni**, per evitare i **ritardi di propagazione** della versione asincrona – ma il principio del conteggio binario è lo stesso.)
Lo stesso circuito, letto in modo diverso, è anche un **divisore di frequenza**: ogni stadio dimezza la frequenza del segnale che lo pilota. Un **orologio digitale**, ad esempio, usa una catena di questi divisori per trasformare un clock molto veloce e preciso in un impulso che avanza esattamente una volta al secondo.
Contatore sincrono, per confronto: esiste anche una variante in cui **tutti i flip-flop condividono** lo **stesso** segnale di clock (invece di essere pilotati a cascata l'uno dall'altro): tutti gli stadi cambiano nello stesso istante, senza i piccoli ritardi di propagazione della versione asincrona. È la soluzione preferita nei circuiti reali ad alta velocità, ma richiede una logica di controllo leggermente più complessa per decidere quando ciascuno stadio deve incrementare.

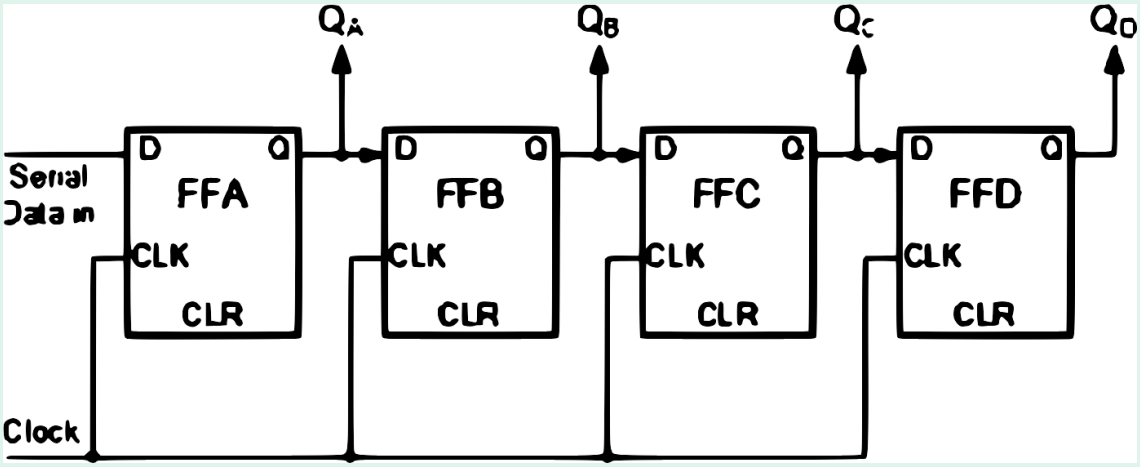
5 Il registro a scorrimento (shift register)

Definizione

Cambiando il collegamento tra i flip-flop D in cascata – l'uscita Q di ciascuno collegata all'**ingresso** D del **successivo**, invece che $\overline{Q}$ al proprio D – si ottiene un **registro a scorrimento** (*shift register*): un dispositivo che fa **avanzare un bit alla volta** lungo la catena a ogni colpo di clock.

Esempio

Il registro a scorrimento a **4 bit** completo, come disegnato alla lavagna: quattro flip-flop D in cascata (FFA, FFB, FFC, FFD). L'uscita Q di ciascuno diventa l'ingresso D del **successivo** (non del proprio, come invece accadeva nel contatore), e tutti **condividono lo stesso** segnale di **Clock**.



La catena completa ha ingresso **Serial Data in** sul primo flip-flop (FFA) e le quattro uscite Q_A, Q_B, Q_C, Q_D **leggibili in parallelo**.

Esempio

Un'analogia utile: un **tubo di cartone** in cui si infilano, una alla volta, palline di due colori diversi (bianche e nere, rappresentano i bit 1 e 0). Ogni volta che si infila una nuova pallina da un'estremità, tutte le altre avanzano di una posizione. Guardando il tubo dall'esterno, si vedono **tutte le palline contemporaneamente**, anche se sono state inserite una alla volta, in sequenza.
Questo è esattamente ciò che fa lo **shift register**: i dati **entrano in serie** (un bit alla volta, su un unico filo SER) e possono essere letti tutti insieme **in parallelo** (su più fili contemporaneamente, le uscite Q_A, Q_B, Q_C, Q_D). Per questo si parla di conversione **seriale** $\rightarrow$ **parallelo**.

6 Il componente 74HC595

Definizione

Il componente commerciale che useremo per realizzare i nostri circuiti è il **74HC595**, uno shift register a **8 bit** con uscite a **3 stati**. Appartiene alla famiglia di circuiti integrati **74** (uso **commerciale**, range di temperatura più limitato) contrapposta alla famiglia **54** (uso militare, range di temperatura più ampio). La sigla **HC** indica la **tecnologia costruttiva CMOS ad alta velocità** (*High-speed CMOS*).

Nota

Le uscite a **3 stati** (*three-state* o *tri-state*) possono assumere, oltre ai normali livelli logico basso (0) e alto (1), un **terzo stato**: **alta impedenza** (non collegato elettricamente). Questo permette di collegare più uscite 3-state allo stesso filo senza rischio di **cortocircuito**, perché solo un dispositivo alla volta "parla" sul filo mentre gli altri restano scollegati.

Esempio

Dati tecnici principali del 74HC595 (dal datasheet Texas Instruments):

- **tensione di alimentazione**: da 2 V a 6 V (si alimenta tranquillamente a 5 V, coerente con la logica TTL/CMOS standard);
- **consumo in bassa potenza**: circa 80 μA (0,00008 A), estremamente contenuto.

Attenzione

Superare la **tensione massima** dichiarata nel datasheet (6 V) rischia di **bruciare il componente**. Alimentarlo con una tensione insufficiente (es. 1 V) non lo danneggia, ma semplicemente **non lo fa funzionare**.

6.1 Package: PTH vs SMD

Definizione

I componenti elettronici si dividono per tipo di montaggio sulla scheda:

- **PTH** (*Pin Through Hole*, a foro passante): i **piedini attraversano un foro** nella scheda e vengono saldati dal lato opposto. È il tipo di componente usato finora in laboratorio (es. sulla breadboard).
- **SMD** (*Surface Mounting Device*, montaggio superficiale): i piedini sono piegati a "**zampa di gabbiano**" e vengono semplicemente appoggiati e saldati sulla superficie della scheda, **senza fori**.

Nota

Il vantaggio dell'SMD è lo **spazio**: non serve riservare un'area della scheda per i **fori di attraversamento**, quindi si possono realizzare circuiti **più compatti e densi**. Lo svantaggio è che la **saldatura SMD** richiede una tecnica diversa da quella PTH, più delicata per via delle dimensioni ridotte dei piedini.

Esempio

Contenitori (*package*) comuni per il 74HC595:

- **DIP** (*Dual Inline Package*): due file di piedini allineati, tipico del PTH. Con l'involucro in ceramica si parla di **CDIP**, con involucro in plastica di **PDIP** (il più comune nei nostri laboratori).
- SOIC, **SOP**: contenitori **SMD**, più piccoli, con passo tra i piedini di 1,27 mm (contro il passo standard di 2,54 mm/100 mil del PTH). Varianti ancora più compatte come **SSOP**/**TSSOP** scendono fino a 0,65 mm di passo.

La sigla precisa del componente da ordinare cambia in base al package: ad esempio `SN54HC595J` per la versione **ceramica**, `74HC595D` per la versione **SOIC**.

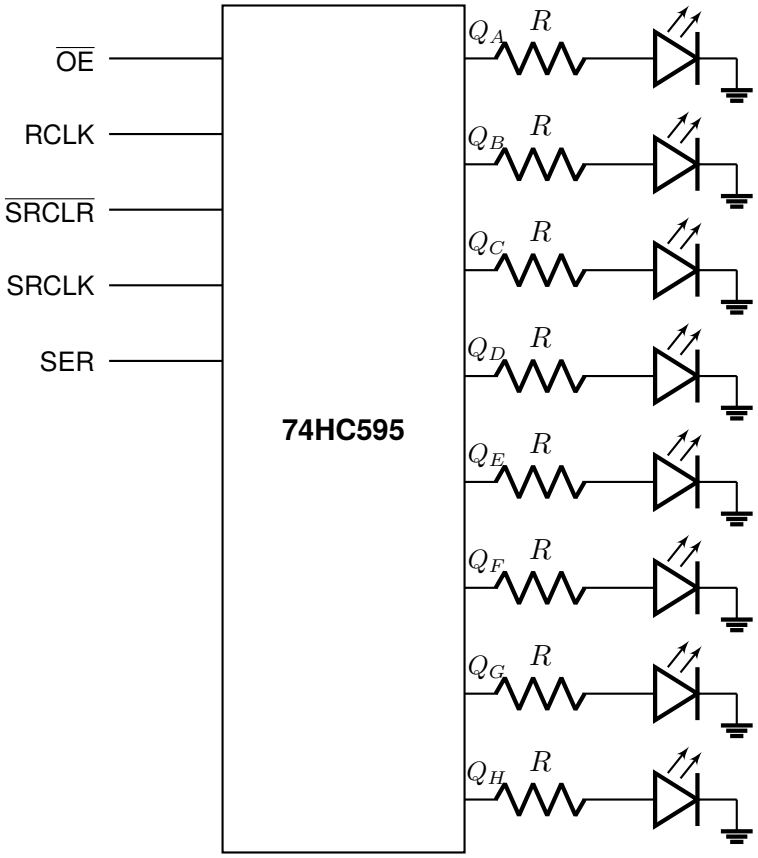
6.2 La piedinatura del 74HC595

Definizione

I piedini di controllo del 74HC595, letti dal **datasheet ufficiale** (Texas Instruments):

- **SER** (piedino 14): l'**ingresso dati seriale**, dove si inseriscono i bit uno alla volta – equivale all'ingresso D del nostro flip-flop D.
- **SRCLK** (piedino 11): il **clock dello shift register** interno – ogni **fronte di salita** fa avanzare di una posizione il bit appena inserito in SER, esattamente come nello schema a flip-flop D visto sopra.
- **RCLK** (piedino 12, *Register Clock*, detto anche *latch clock*): un **secondo** registro, separato dallo shift register interno, che **copia tutti gli 8 bit** sulle uscite solo quando riceve un proprio **fronte di salita**.

- **SRCLR** (piedino 10, *Shift Register Clear*, negato): resetta tutto lo shift register interno. Nel nostro uso resta sempre collegato a $+5$ V (livello alto, "non attivo": essendo negato, si attiva solo quando portato a livello basso).
- **OE** (piedino 13, *Output Enable*, negato): abilita o disabilita le uscite a 3 stati. Resta sempre collegato a GND (livello basso, "sempre abilitato").



Nota

Il motivo per cui il 74HC595 ha **due** registri distinti (lo shift register pilotato da SRCLK, e il **registro di uscita** pilotato da RCLK) è evitare un effetto indesiderato: se le uscite si aggiornassero a ogni singolo colpo di SRCLK mentre si inseriscono i bit uno alla volta, si vedrebbe visivamente lo **scorrimento** del dato lungo i LED collegati, invece di un cambiamento istantaneo e pulito. Separando i due clock, tutti gli 8 bit vengono prima caricati nello shift register (SRCLK), e solo alla fine, con un singolo colpo di RCLK, vengono **pubblicati tutti insieme** sulle uscite: il cambiamento appare istantaneo e "asincrono" rispetto al caricamento.

Esempio

La sequenza di comandi per pilotare il 74HC595 da un **microcontrollore** (es. Arduino), per ciascuno degli 8 bit da trasmettere:

1. Scrivere il valore del bit (0 o 1) sul piedino SER.
2. **Generare un fronte di salita** su SRCLK (livello alto, poi livello basso): questo fa **avanzare il bit** nello shift register interno.

3. Ripetere per tutti gli 8 bit, dal primo all'ultimo.
4. Solo alla fine, dopo aver caricato tutti gli 8 bit, **generare** un singolo **fronte di salita** su RCLK: questo **pubblica tutti i dati** sulle uscite contemporaneamente, senza **scorrimento visibile**.

Nel nostro uso, SRCLR resta sempre collegato a $+5$ V e OE resta sempre collegato a GND: gli **unici tre piedini** che il microcontrollore deve effettivamente pilotare sono SER, SRCLK e RCLK.

7 Cosa ricordare di questa lezione

- Un circuito **combinatorio** ha l'uscita che dipende solo dagli ingressi attuali; un circuito **sequenziale** dipende anche dallo **stato precedente**, grazie alla **retroazione**.
- Il **latch SR** (due porte NOR incrociate) è la prima cella di memoria: mantiene lo stato con $S = R = 0$, ma ha una combinazione **non ammessa** ($S = R = 1$).
- Il **flip-flop D** risolve questo problema: aggiorna l'uscita **solo** al **fronte di salita** del clock, funzionando come una **macchina fotografica** dell'ingresso D.
- Collegando flip-flop D in cascata con $\overline{Q} \to D$ si ottiene un **contatore** (in versione *asincrona*, ogni stadio pilotato dal $\overline{Q}$ dello stadio precedente, o *sincrona*, tutti i CLK in comune) – alla base del ***program counter*** e dei **divisori di frequenza**; con $Q \to D$ e CLK comune a tutti gli stadi si ottiene invece un **registro a scorrimento** (conversione seriale$\to$parallelo).
- Il **74HC595** è uno shift register a **8 bit** con uscite 3-state, pilotato da **tre segnali**: **SER** (**dato**), **SRCLK** (**avanza lo shift register**), **RCLK** (**pubblica il risultato** in uscita, evitando lo scorrimento visibile).
- I componenti si distinguono per montaggio **PTH** (a foro, es. DIP) o **SMD** (superficiale, es. SOIC/SOP), con **package e sigle diverse** in base alla scelta.